

Tracing Circuits in Qwen3: A Faithful Reimplementation and Reproduction of Attribution Graphs*

Zixuan Wang
zw499@cam.ac.uk

July 12, 2026

Abstract

Attribution graphs decompose a language model’s forward pass on a single prompt into causal, weighted interactions between sparse dictionary features, and have been used to map the “biology” of a frontier model. We reimplement the full methodology from scratch for the Qwen3 model family — local replacement models, batched edge attribution, influence-based pruning, and constrained-patching interventions — using the open-source circuit-tracer library only to load pretrained per-layer transcoders. The reimplementation is verified numerically equivalent to circuit-tracer: identical node sets, matching edge weights, identical pruned graphs, and matching constrained-steering effects on shared inputs. We then reproduce the multilingual-antonyms case study of Lindsey et al. [3] on Qwen3-4B — shared cross-lingual circuitry and the paper’s operation, operand, and language swap interventions — following its protocols, and add a same-recipe cross-scale comparison (4B vs. 8B). The circuit structure reproduces; the operand swap reproduces at 1.5–12× the paper’s steering strengths; the language swap reproduces in two of three directions at the paper’s own scale; and the operation swap does not reproduce under the protocol’s frozen attention patterns — a probe that lets patterns re-form recovers the paper’s exact outcome tokens, identifying the channel as QK-mediated, the same interaction the paper flags as invisible to its method. The cross-scale overlap-growth claim does not survive a register-diverse evaluation corpus.

1 Introduction

Mechanistic interpretability aims to reverse-engineer the computations of neural networks into human-understandable algorithms [4, 5]. For large language models a central obstacle is that the natural unit of computation is not the neuron: models appear to represent many more concepts than they have neurons, storing them in *superposition*, so individual neurons are polysemantic and resist interpretation [6]. Explaining a behaviour mechanistically therefore requires both a better feature basis and a way to trace, on concrete prompts, how those features causally interact.

Dictionary learning addresses the first requirement. Sparse autoencoders (SAEs) decompose activations into an overcomplete basis of sparsely-firing, substantially more monosemantic features [7, 8], scale to frontier models [9], and benefit from discontinuous activation functions such as JumpReLU [10]. An SAE, however, only re-describes the representation at one point in the network: it says what is encoded, not how it is computed. *Transcoders* [11] instead train the same sparse dictionary architecture to imitate an MLP’s input–output map, so that each feature has both an input action (its encoder row) and an output action (its decoder column) and the MLP’s computation factors through an interpretable sparse bottleneck. Cross-layer variants let one feature write to all subsequent layers [1].

*Code: `llm-circuits` repository. Reference method: Ameisen et al. [1]; reference implementation: Hanna et al. [2].

The second requirement — tracing interactions — has traditionally been met with causal patching. Manual activation patching over attention heads and MLPs isolated, e.g., the indirect-object-identification circuit in GPT-2 [12]; ACDC automated the search [13]; attribution patching replaced interventions with cheap gradient approximations [14]; and sparse feature circuits lifted patching from model components to SAE features [15]. *Attribution graphs* [1] sharpen this line of work into a per-prompt causal graph over transcoder features. Replacing MLPs with transcoders, freezing attention patterns and normalisation denominators at their on-prompt values, and inserting error nodes yields a *local replacement model* that reproduces the underlying model’s forward pass exactly while being linear in the feature activations; *direct* effects between features are then exact linear coefficients obtained by backpropagation rather than approximations, and the resulting dense graph is pruned to an interpretable subgraph whose hypotheses are validated by steering interventions on the real model. The companion paper applies the method across dozens of behaviours of Claude 3.5 Haiku [3], and the circuit-tracer library [2] open-sources the pipeline for open-weight models. Together these form the reference for the present work.

This report makes two contributions. **First**, we reimplement the entire pipeline from scratch for HuggingFace Qwen3 models [16]: capture of frozen constants, the local replacement forward pass, batched edge attribution via `autograd`, influence-based graph pruning, an interactive graph explorer, and constrained-patching feature interventions. `circuit-tracer` is used *only* to load pretrained per-layer transcoders [17]; no attribution, pruning, or intervention machinery is imported. A dedicated verification suite establishes numerical equivalence with `circuit-tracer` on shared inputs — identical node sets, edge weights matching to floating-point noise, identical influence rankings and pruned graphs, and matching constrained-steering deltas (Appendix A). **Second**, we reproduce the *multilingual antonyms* case study of Lindsey et al. [3] on Qwen3-4B: a shared cross-lingual “say-large” core with language-specific input and output stages, the paper’s operation/operand/language swap interventions under its constrained-patching protocol, and cross-lingual feature overlap as a function of scale, compared against Qwen3-8B with same-recipe transcoders. Where Qwen3 forces protocol adaptations — chat templates for an instruct model, steering strengths at which a 4B model responds only weakly — we document the adaptation explicitly and separate model differences from method failures.

Section 2 presents the method; Section 3 reports the verification and the reproduction; Section 4 discusses what transfers across models and what does not.

2 Method

Throughout, the underlying model is a pre-norm decoder-only transformer with L layers. We write $\mathbf{x}_c^\ell \in \mathbb{R}^d$ for the residual stream at token position c before layer ℓ ’s MLP, $\tilde{\mathbf{x}}_c^\ell$ for the normalised MLP input,¹ and $\mathbf{y}_c^\ell = \text{MLP}^\ell(\tilde{\mathbf{x}}_c^\ell)$ for the MLP output added back into the stream. Logits are $\text{logit}_t = \mathbf{u}_t^\top \text{norm}(\mathbf{x}^{L+1})$ with unembedding rows $\mathbf{u}_t$.

2.1 Transcoders: sparse dictionaries for MLP computation

An SAE learns an overcomplete sparse code $\mathbf{f}(\mathbf{x}) = \sigma(W_{\text{enc}}\mathbf{x} + \mathbf{b}_{\text{enc}})$ with linear decoder $\hat{\mathbf{x}} = W_{\text{dec}}\mathbf{f} + \mathbf{b}_{\text{dec}}$, trained to reconstruct its own input under a sparsity penalty. A *transcoder* keeps this architecture but changes the target: it is trained to predict the MLP’s *output* from the MLP’s *input*, so that the learned features mediate the layer’s computation rather than merely re-encode its input. In the per-layer form (PLT) used in this work, layer ℓ has feature activations and reconstruction

$$\mathbf{a}_c^\ell = \text{JumpReLU}(W_{\text{enc}}^\ell \tilde{\mathbf{x}}_c^\ell + \mathbf{b}_{\text{enc}}^\ell), \quad \hat{\mathbf{y}}_c^\ell = W_{\text{dec}}^{\ell \top} \mathbf{a}_c^\ell + \mathbf{b}_{\text{dec}}^\ell, \quad \text{JumpReLU}(z)_i = z_i \mathbb{I}[z_i > \theta_i], \quad (1)$$

¹In Qwen3, RMSNorm: $\tilde{\mathbf{x}} = \mathbf{x} \odot \sigma(\mathbf{x}) \odot \gamma$ with $\sigma(\mathbf{x}) = \sqrt{\|\mathbf{x}\|^2/d + \varepsilon}$.

i.e. a linear map on either side of a hard sparse gate. The reference method’s default is the *cross-layer* transcoder (CLT), in which a feature encoded at layer ℓ' writes to the MLP outputs of *all* later layers through separate decoders, $\hat{\mathbf{y}}_c^\ell = \sum_{\ell' \leq \ell} W_{\text{dec}}^{\ell' \rightarrow \ell \top} \mathbf{a}_c^{\ell'} + \mathbf{b}_{\text{dec}}^\ell$; the PLT is the special case $W_{\text{dec}}^{\ell' \rightarrow \ell} = 0$ for $\ell' \neq \ell$. Training minimises reconstruction error with a tanh sparsity penalty,

$$\mathcal{L} = \sum_{\ell=1}^L \|\hat{\mathbf{y}}^\ell - \mathbf{y}^\ell\|_2^2 + \lambda \sum_{\ell,i} \tanh(c \|W_{\text{dec},i}^\ell\| a_i^\ell), \quad (2)$$

with the JumpReLU thresholds θ trained through a straight-through estimator. The pretrained Qwen3 transcoders used here are per-layer JumpReLU dictionaries with 163,840 features per layer and no skip connection [17].

2.2 Attribution-graph generation

Local replacement model. Substituting every MLP by its transcoder yields the *replacement model*, which only approximates the underlying model. The attribution graph is instead built on the *local replacement model* for a specific prompt p : (i) MLPs are replaced by transcoders; (ii) attention patterns and normalisation denominators are *frozen* at the values they take in the underlying model’s forward pass on p , making attention and normalisation linear maps of the residual stream; and (iii) an *error node* is added at every (position, layer),

$$\mathbf{e}_c^\ell = \mathbf{y}_c^\ell - \hat{\mathbf{y}}_c^\ell, \quad (3)$$

injected as a constant so that every activation and logit of the local replacement model *exactly* equals the underlying model’s. By construction the only remaining nonlinearities are the JumpReLU gates on feature pre-activations: between any feature’s (constant-scaled) output and any downstream pre-activation, the map is affine.

Nodes. The graph has four node types: one *embedding* node per prompt token; one *feature* node per active feature instance $s = (c_s, \ell_s, i_s)$ with activation $a_s > 0$; one *error* node per (position, layer); and *logit* nodes for the smallest set of candidate output tokens whose cumulative probability reaches 0.95, capped at 10, at the final position. Embedding and error nodes have no incoming edges; logit nodes have no outgoing edges.

Edges. Edge weights are the *direct* linear effects of source on target within the local replacement model. Let $\bar{J}_{(c_s, \ell) \rightarrow (c_t, \ell_t)}$ denote the Jacobian of the residual stream at (c_t, ℓ_t) with respect to the stream at (c_s, ℓ) computed with stop-gradients on all MLP/transcoder outputs, attention patterns, and normalisation denominators — i.e. the sum over all paths through the residual stream and the frozen attention OV circuits, but through no other feature’s nonlinearity. For a feature source s and a feature target t with encoder row $W_{\text{enc}, i_t}^{\ell_t}$, the edge weight is

$$A_{s \rightarrow t} = a_s \sum_{\ell_s \leq \ell < \ell_t} (W_{\text{dec}, i_s}^{\ell_s \rightarrow \ell})^\top \bar{J}_{(c_s, \ell) \rightarrow (c_t, \ell_t)} W_{\text{enc}, i_t}^{\ell_t}, \quad (4)$$

which for the PLT collapses to the single $\ell = \ell_s$ term. Embedding and error sources use their output vectors ($\mathbf{E}_{\text{tok}}$ and $\mathbf{e}_c^\ell$) in place of $a_s W_{\text{dec}, i_s}^\ell$. For a logit target the input vector is the *vocabulary-demeaned* unembedding direction $\mathbf{u}_t - \bar{\mathbf{u}}$, applied at the (frozen-)normalised final residual — the demeaned logit is the component that affects the softmax, which is invariant to uniform shifts. Because the local replacement model is affine in the source outputs, incoming edges are *complete*: the pre-activation of any feature node satisfies $h_t = \sum_s A_{s \rightarrow t} + \text{const}$, where the constant collects bias-path terms ($\mathbf{b}_{\text{enc}}$, $\mathbf{b}_{\text{dec}}$) that are not represented as nodes. Attention QK circuits are deliberately outside the graph: nodes influence one another through frozen attention outputs, but effects *on* the patterns themselves are not modelled.

Computation. Edges are computed target-by-target with one backward pass per batch of targets: the target’s input vector is injected as the gradient seed at its (layer, position), the backward pass runs through the frozen-linear graph, and every source’s edge weight is the inner product of the resulting gradient at the source’s write location with the source’s output vector, as in Eq. (4). The cost is linear in the number of attributed targets and independent of the number of sources; on dense prompts, targets can be selected adaptively in decreasing order of an incrementally re-estimated influence on the logits, so that the most explanatory feature nodes are attributed first.

2.3 Graph pruning

Raw graphs contain 10^4 – 10^6 edges, most irrelevant to the output. Pruning keeps the subgraph that carries the bulk of the influence on the logits. Let A be indexed $[t, s]$ (rows = targets) and define the normalised unsigned adjacency

$$\hat{A}_{ts} = \frac{|A_{ts}|}{\sum_{s'} |A_{ts'}|}, \quad B = \sum_{k \geq 1} \hat{A}^k = (I - \hat{A})^{-1} - I, \quad (5)$$

where the Neumann series converges because the graph is a DAG ($\hat{A}$ is nilpotent); B_{tv} sums the strengths of all directed paths $v \rightarrow t$, a path’s strength being the product of its normalised edge weights. Each node v receives the probability-weighted logit influence

$$I_v = \sum_{t \in \text{logits}} p_t B_{tv}, \quad (6)$$

with p_t the model’s output probability for logit node t . **Node pruning** keeps the minimal set of nodes, in decreasing I_v , whose cumulative share of $\sum_v I_v$ reaches $\tau_{\text{node}} = 0.8$; embedding and logit nodes are always kept.² **Edge pruning** recomputes $\hat{A}'$ and I' on the pruned graph and scores each surviving edge by the influence it feeds into its target,

$$s_{ts} = \hat{A}'_{ts} (I'_t + p_t), \quad (7)$$

($p_t = 0$ for non-logit targets, and $I'_t = 0$ for logit nodes, so the parenthesis is the target’s score in either case), again keeping the minimal set with cumulative share $\tau_{\text{edge}} = 0.98$. Finally, internal nodes left dangling — features without both an incoming and an outgoing kept edge, error nodes without an outgoing one — are removed iteratively. Typical graphs shrink by roughly an order of magnitude in nodes and two in edges while retaining $\approx 80\%$ of the logit influence.

2.4 Interventions

Attribution graphs are hypotheses; they are validated by steering features in the *underlying* model and checking that downstream features and logits move as the graph predicts. Steering feature s multiplicatively by M — $a_s \mapsto M a_s$, so $M=0$ ablates and $M=-1$ flips the sign³ — adds the corresponding decoder delta to every MLP output the feature writes to:

$$\Delta \mathbf{y}_{c_s}^\ell = (M - 1) a_s W_{\text{dec}, i_s}^{\ell_s \rightarrow \ell}, \quad \ell_s \leq \ell \leq \ell_e. \quad (8)$$

Under **constrained patching** over a layer range $[\ell_s, \ell_e]$, all MLP outputs within the range are pinned to their clean recorded values before the deltas are added, so the intervention itself is the *only* change to any MLP output in the range (no second-order effects within it, matching the

²The paper’s prose also protects error nodes; the reference implementation (and ours, verified equivalent to it) lets error nodes be pruned.

³Our code parameterises the same operation by the additive multiple $m = M - 1$ of the clean activation ($m=-1$ ablate, $m=-2$ flip); circuit-tracer’s API takes the absolute target value $M a_s$. All strengths in this report are quoted in the paper’s multiplicative M convention.

graph’s direct-effect semantics); computation resumes normally after ℓ_e . Attention patterns are frozen to their clean values throughout, consistent with the graph’s frozen-attention assumption; normalisation denominators are frozen only when the range spans the whole model, in which case the intervention measures a pure direct (linear) effect on the logits. The alternative, **iterative patching**, lets downstream features re-encode their perturbed inputs and take new values, propagating knock-on effects through the real MLPs at the price of compounding reconstruction errors. Effects are read out both as changes in the output distribution and as downstream feature activations re-encoded from the perturbed forward pass, reported as percentages of their clean baselines. Because feature dictionaries are imperfect (reconstruction error, incomplete feature recruitment), steering at $|M|$ beyond the observed activation is often required before behavioural effects appear; the paper typically suppresses source features to $M = -1$ or beyond and drives donor features at several times their donor-prompt activation.

Reimplementation. Our implementation realises the above for HuggingFace Qwen3 checkpoints with per-layer transcoders: a capture pass records attention patterns, RMSNorm denominators and MLP outputs; hooks then replace MLPs with transcoder reconstructions plus constant error nodes, substitute frozen-pattern attention (V/O only) and frozen-denominator norms; batched **autograd** computes Eq. (4) for hundreds of targets per backward pass. Pruning follows Section 2.3 exactly, and interventions follow circuit-tracer’s constrained semantics. An eleven-check verification suite plus three steering-parity cases establish numerical equivalence with circuit-tracer: node sets, edge weights, influence rankings, pruned graphs, and constrained-steering deltas all agree to floating-point noise (Table 2, Appendix A).

3 Experiments and Results

3.1 Prompts, behaviour, supernodes, and attribution graphs

All experiments use Qwen3-4B (36 layers, bf16) with pretrained per-layer JumpReLU transcoders (163,840 features per layer) [17] on a single A100-80GB; notebooks seed all RNGs and pin cuDNN to deterministic kernels at startup, and re-executions reproduce every reported number exactly.

Prompts and behaviour. The paper’s prompts are raw completions ending on a content-bearing open quote; we run them exactly, with one attention-sink token prepended:

```
<|endoftext|>The opposite of "small" is "  
first generated token: large
```

Because Qwen3-4B is instruction-tuned, a chat-template adaptation runs alongside (thinking disabled; the answer is again the first generated token):

```
<|im_start|>user  
What is the opposite of "small"? Reply with only the word,  
nothing else.<|im_end|>  
<|im_start|>assistant  
<think>  
  
</think>  
  
first generated token: large
```

The antonym task is intact in both formats and all three languages (large / grand / 大); all ten prompts (three antonym recipients plus the English synonym and hot donors, per format) and their answers are tabulated in Appendix B.

placeholder — example reviewed features (labels, top output logits, top-activating dataset examples); screenshot to be provided

Figure 1: Examples of reviewed supernode members: each feature’s top output logits and top-activating dataset examples — the evidence the semantic selection matches against.

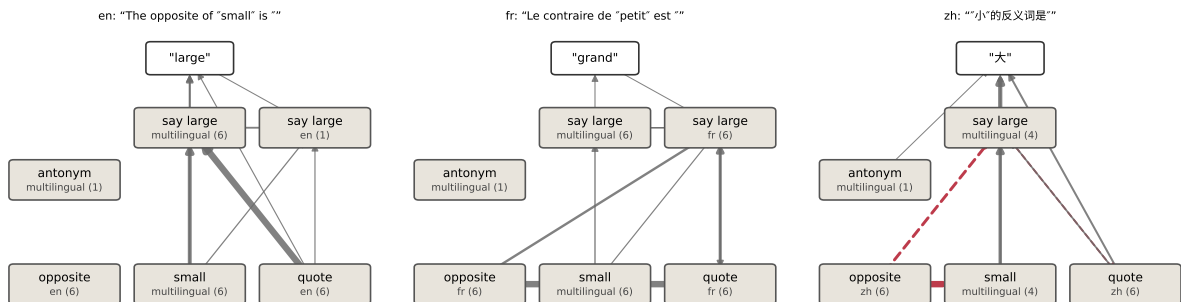


Figure 2: Simplified attribution graphs for the three raw antonym prompts. Boxes are the reviewed supernodes (member count shown); arrow widths are proportional to the aggregated signed edge weight between groups in the pruned graph (red dashed: net-negative). The paper’s shared-core skeleton appears in all three languages.

Supernodes and attribution graphs. Supernodes are selected by *semantics*, *not influence rank*: a scan over the pruned graphs matches each feature’s top output logits and top-activating input examples against per-concept lexicons, ranks candidates earliest-layer-first, and caps each group at six members; the chat and raw formats get fully separate selections, frozen in reviewed artifacts with per-graph activations and node positions per member, and enforced disjoint by (layer, feature). Figure 1 shows the evidence this selection matches against for example members. Figure 2 shows the resulting simplified graphs for the three raw antonym prompts: the paper’s skeleton — language-specific *opposite* and *quote* input groups, a shared multilingual *small* group, and a shared *say large* stage feeding the answer — is present in all three languages, with 107 (chat) and 67 (raw) individual features appearing in all three languages’ pruned graphs. One structural difference is visible immediately: the groups sit late (the raw *antonym* group has a single L34 member; *quote* members span L23–L34), where the paper describes early detection features. The chat counterpart of Figure 2 and full interactive-explorer views of all six antonym pages are in Appendix C.

3.2 Swap interventions

All swaps run the paper’s constrained-patching protocol (Section 2.4): source members are steered to a negative multiple of their clean activation at their own node positions, donors are injected at an absolute multiple of their donor-graph activation, the patch end layer ℓ is swept and the most effective value reported, and supernode readouts are means of per-feature ratios with rows at layers $\leq \ell$ pinned by the protocol. Strength ladders extend tenfold beyond the paper’s endpoints along the same source–donor coupling (the paper endpoint is an exact, marked grid point), so paper-strength readings and what-it-takes readings come from one run. Figure 3 summarises all

three swaps.

Operand swap (small $\rightarrow$ hot; paper: $-0.5\times/+1.5\times$ gives cold / f[roid] / 冷). This reproduces in *all six* format-language configurations, but nothing moves at the paper’s endpoint: crossovers sit at $2.25\text{--}4.5\times$ (chat), $5.25\text{--}6\times$ (raw en/fr), and $18\times$ (raw zh), with endpoint outputs cold 0.81–0.93, f 0.73–0.97, 冷 0.998. Where readouts are readable the paper’s push-pull appears at amplitude — *say cold* recruited to 18–65% of its donor-prompt level, *say large* suppressed to 4–26%. The paper’s own caveat that required strengths are “quite high” transfers with a larger constant.

Language swap (quote_x $\rightarrow$ quote_y; paper: the output language changes, operation and operand preserved). Two of three directions reproduce at the paper’s own scale: en $\rightarrow$ zh yields 大 at 0.636 (crossover $5.25\times$), and fr $\rightarrow$ en turns the output English — *big* at 0.753, the paper’s exact token (the figure’s curve tracks the model’s own answer “large” and so under-reports this flip). zh $\rightarrow$ fr never lands. Beyond $\sim 9\times$ both working directions *degrade* (junk or wrong-language intrusions): unlike the operand swap, this handle saturates. The one readable annotation matches the paper’s pattern (*say large (fr)* $\rightarrow$ 0% under fr $\rightarrow$ en). Because Qwen3’s pruned raw graphs keep no early quote-position nodes, the causal handle here is *late* (L23–L34) rather than the paper’s early detection features.

Operation swap (antonym $\rightarrow$ synonym; paper: language-appropriate synonyms top-1, crossover $\approx 4\times$). This never crosses — at the paper’s endpoint, at $2.5\times$ it, or at $5\times$ it ($\pm 29/30\times$), in either format. The chat configuration has full protocol room (ℓ swept over [13, 35]) and readable annotations, which show the mechanism moving in the right direction but far too weakly: *say small* recruits to at most 37% of donor level and the output drifts to adjacent semantics (*medium* at 0.70) without executing the operation. A probe isolates the cause: running the identical configuration in fully-propagating mode with attention patterns *frozen* reproduces the flat null, while letting patterns *re-form* lands the paper’s exact outcome tokens — *little* at 0.735 (en, crossover $6.75\times$) and *pet* at 0.515 (fr, $4.5\times$) (Figure 3, bottom left). The swap’s channel is QK-mediated — the very interaction the paper reports for Haiku as “mediated by changing where attention heads attend” and invisible to its method — and constrained patching freezes attention patterns by construction, so no constrained configuration can land it. In sum, the three swaps have three characters on Qwen3-4B: the operand swap is the paper’s result with a bigger constant; the language swap is the paper’s result at the paper’s scale, with a ceiling; the operation swap is blocked by the protocol’s frozen-QK assumption meeting a QK-mediated circuit.

3.3 Cross-lingual overlap, scale, and the default language

Following the paper, we compute the per-layer intersection-over-union of the feature sets active anywhere in the context for translated paragraph pairs, against an unrelated-pair baseline, on 18 register-diverse parallel paragraphs (authored in English and translated to French and Chinese — the paper’s own corpus recipe). The paper’s shape reproduces cleanly (Figure 4): baseline-subtracted overlap is low at both ends and peaks mid-network, with mid-third means en-fr 0.198 > en-zh 0.175 > fr-zh 0.142 — the paper’s pair ordering. The paper’s *scale* claim does not reproduce on a same-recipe transcoder pair (4B vs. 8B, both 36 layers and 163,840 features/layer): mid-third changes are +6% / −3% / 0%, with no emphasis on the non-alphabet-sharing pairs the paper highlights. Finally, the paper’s Claude-specific claim that English is the mechanistic default: the mean direct decoder effect of the shared *say-large* features on the three answer tokens is zh 0.766 > en 0.664 $\gg$ fr 0.309 — *some* language is privileged, but for EN/ZH-trained Qwen3 the role is shared between English and Chinese.

3.4 Verdicts

Table 1 condenses the comparison. The paper’s central structural claims transfer; its interventions transfer in proportion to how much their causal route depends on machinery the protocol holds

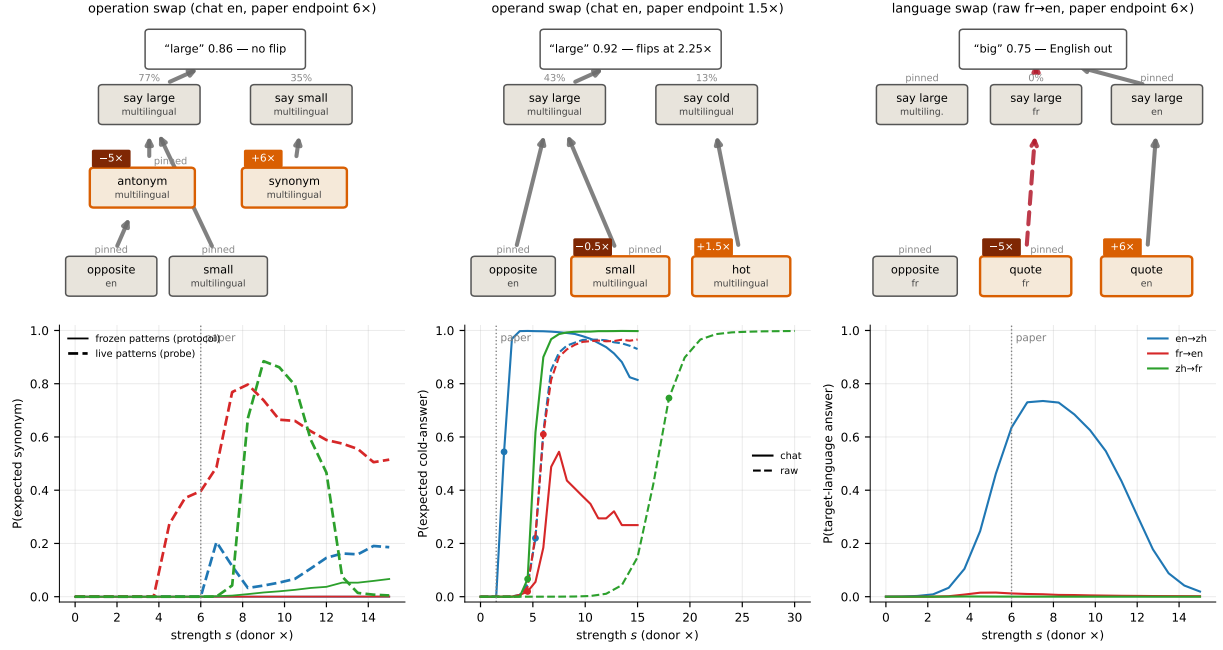


Figure 3: Swap interventions. *Top*: intervention diagrams with measured supernode readouts (grey percentages; “pinned” = at or below the patch end layer ℓ , clamped by the protocol); badges give the paper’s steering multiples. *Bottom*: strength ladders, $P(\text{expected answer})$ vs. donor multiple s , with the paper endpoint dotted. Left: the operation swap under the constrained protocol (solid, flat) vs. the live-attention probe (dashed) — the QK-mediated channel. Middle: the operand swap crosses in all six configurations (dots mark crossovers; raw-zh at 18×). Right: the language swap peaks near the paper’s own 6× and saturates; fr→en’s curve tracks “large” while the actual output is the paper’s token “big” (0.753 at 6×).

fixed — fully for the MLP-pathway operand swap (given strength), partially for the direct-path language swap, and not at all for the QK-mediated operation swap.

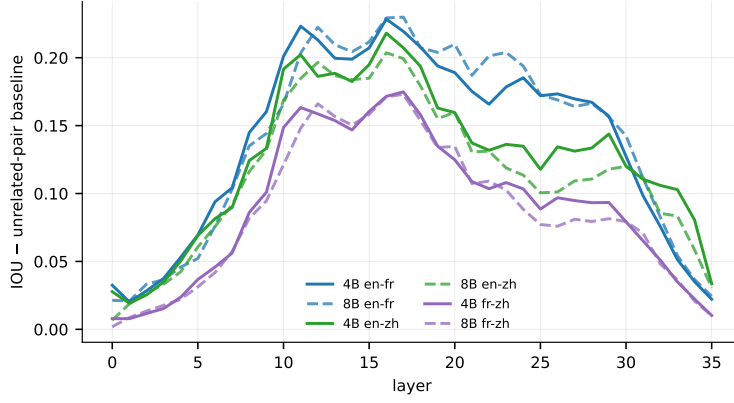


Figure 4: Baseline-subtracted cross-lingual feature overlap by layer (solid: 4B; dashed: 8B, same-recipe transcoders). The paper’s inverted-U and pair ordering reproduce; its cross-scale growth does not.

paper claim (Claude 3.5 Haiku)	Qwen3-4B measurement	verdict
behaviour big / grand / 大	large / grand / 大, both formats	reproduced
shared multilingual core	107 (chat) / 67 (raw) features in all three pruned graphs	reproduced
operand swap at $-0.5\times/+1.5\times$	flips in all six configurations, crossovers $2.25\text{--}18\times$	reproduced*
language swap, three directions	2/3 at the paper’s scale; saturates past $\sim 9\times$	partial
operation swap at $-5\times/+6\times$	no flip $\leq 30\times$ frozen; flips once patterns re-form	not reproduced [†]
overlap: mid-network peak, en-fr highest	$0.198 > 0.175 > 0.142$, ends ≤ 0.03	reproduced
English as mechanistic default	zh $0.766 >$ en $0.664 \gg$ fr 0.309	model difference
scale: 8B > 4B overlap	$+6\% / -3\% / 0\%$	not reproduced

Table 1: Verdicts vs. Lindsey et al. [3]. *At $1.5\text{--}12\times$ the paper’s strengths. [†]Blocked by frozen attention patterns; the channel is QK-mediated (Section 3.2).

4 Discussion

[Placeholder — to be written; budgeted at ≈ 350 words.]

References

- [1] Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, et al. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- [2] Michael Hanna, Mateusz Piotrowski, Jack Lindsey, and Emmanuel Ameisen. circuit-tracer: An open-source library for attribution graphs, 2025. <https://github.com/decoderresearch/circuit-tracer>, v0.3.1.
- [3] Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, et al. On the biology of a

- large language model. *Transformer Circuits Thread*, 2025. <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.
- [4] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 2020. <https://distill.pub/2020/circuits/zoom-in>.
 - [5] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. <https://transformer-circuits.pub/2021/framework>.
 - [6] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition. *Transformer Circuits Thread*, 2022. https://transformer-circuits.pub/2022/toy_model.
 - [7] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nicholas L. Turner, Cem Anil, Carson Denison, Amanda Askell, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features>.
 - [8] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *International Conference on Learning Representations (ICLR)*, 2024.
 - [9] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, et al. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. *Transformer Circuits Thread*, 2024. <https://transformer-circuits.pub/2024/scaling-monosemanticity>.
 - [10] Senthooran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár, and Neel Nanda. Jumping ahead: Improving reconstruction fidelity with jumprelu sparse autoencoders. *arXiv preprint arXiv:2407.14435*, 2024.
 - [11] Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. Transcoders find interpretable LLM feature circuits. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
 - [12] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations (ICLR)*, 2023.
 - [13] Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
 - [14] Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery. In *Proceedings of the 7th BlackboxNLP Workshop*, 2024.
 - [15] Samuel Marks, Can Rager, Eric J. Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. In *International Conference on Learning Representations (ICLR)*, 2025.
 - [16] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

- [17] Michael Hanna. Per-layer transcoders for the qwen3 model family, 2025. HuggingFace model repositories `mwhanna/qwen3-{0.6b,4b,8b}-transcoders`.

A Verification against circuit-tracer

Two scripts certify the reimplementaion against circuit-tracer on shared inputs (Qwen3-0.6B, fp32). The graph suite checks forward exactness of the local replacement model, an exact linearity identity for the frozen-linear backward, parity of salient logit selection, and a full cross-check against `circuit_tracer.attribute()`; the intervention script compares constrained-steering *deltas* across the two implementations for the three primitives the reproduction uses (a single negative steer, a two-layer coupled suppression stack, and an absolute-value donor injection). All eleven graph checks and all three steering cases pass (Table 2).

check	result
forward exactness (local replacement vs. model)	$\max \Delta_{\text{logits}} = 1.2 \times 10^{-5}$
linearity identity of the frozen backward	max error 1.7×10^{-5}
200 emitted edge weights vs. plain <code>autograd</code>	$\max \Delta = 3.7 \times 10^{-5}$
salient-logit selection	identical tokens and probabilities
feature node sets	Jaccard 1.0
edge weights on 2.44M shared pairs	cosine = Pearson = 1.0000
influence ranking	Spearman 1.0000
pruned node sets (0.8/0.98)	identical (337 = 337)
steering deltas (3 intervention primitives)	$\max \Delta_{\text{ours}} - \Delta_{\text{CT}} = 6 \times 10^{-5}$, cosine > 0.99

Table 2: Numerical verification of the reimplementaion against circuit-tracer.

B The ten prompts and their answers

graph	format	prompt (chat: the user line of the templated input; raw: sink token prepended)	answer
antonym_en	chat	What is the opposite of "small"? Reply with only the word, nothing else.	large
antonym_fr	chat	Quel est le contraire de "petit" ? Réponds avec un seul mot, rien d'autre.	Grand
antonym_zh	chat	"小"的反义词是什么? 只回答一个词, 不要其他内容。	大
synonym_en	chat	Give one synonym of "small". Reply with only the word, nothing else.	tiny
hot_en	chat	What is the opposite of "hot"? Reply with only the word, nothing else.	cold
raw_antonym_en	raw	The opposite of "small" is "	large
raw_antonym_fr	raw	Le contraire de "petit" est "	grand
raw_antonym_zh	raw	"小"的反义词是"	大
raw_synonym_en	raw	A synonym of "small" is "	tiny
raw_hot_en	raw	The opposite of "hot" is "	cold

Table 3: The ten graph prompts and the model’s answer (first generated token). Not shown: the French and Chinese synonym behaviour used as operation-swap targets in Section 3.2 is degenerate — the model echoes the operand (`petit` $\rightarrow$ `pet`, `小` $\rightarrow$ `小`) in both formats — and the hot-antonym targets are `cold` / `F · f(roid)` / `冷`.

C Attribution graphs: chat format and explorer snapshots

Figure 5 is the chat-format counterpart of Figure 2 (chat pages have no quote-position nodes, so the *quote* groups exist only on the raw side). Full attribution graphs (all pruned nodes and edges, with the reviewed supernodes pre-loaded as groups) are best inspected in the interactive explorer shipped with the repository; the snapshots below show the six antonym pages.

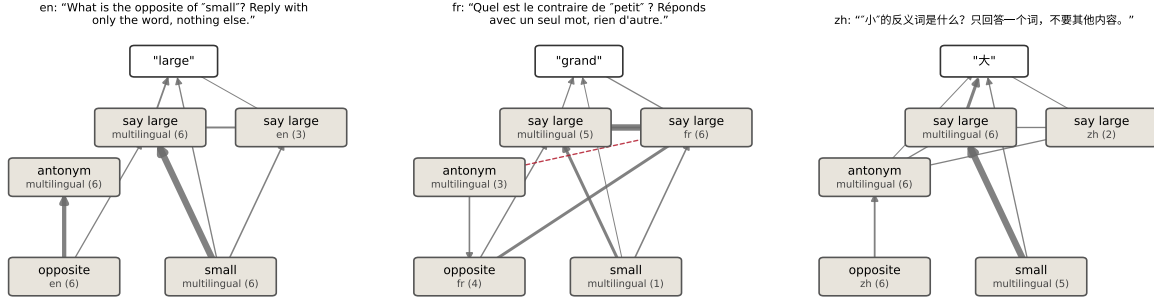


Figure 5: Simplified attribution graphs for the three chat-format antonym prompts (conventions as in Figure 2).

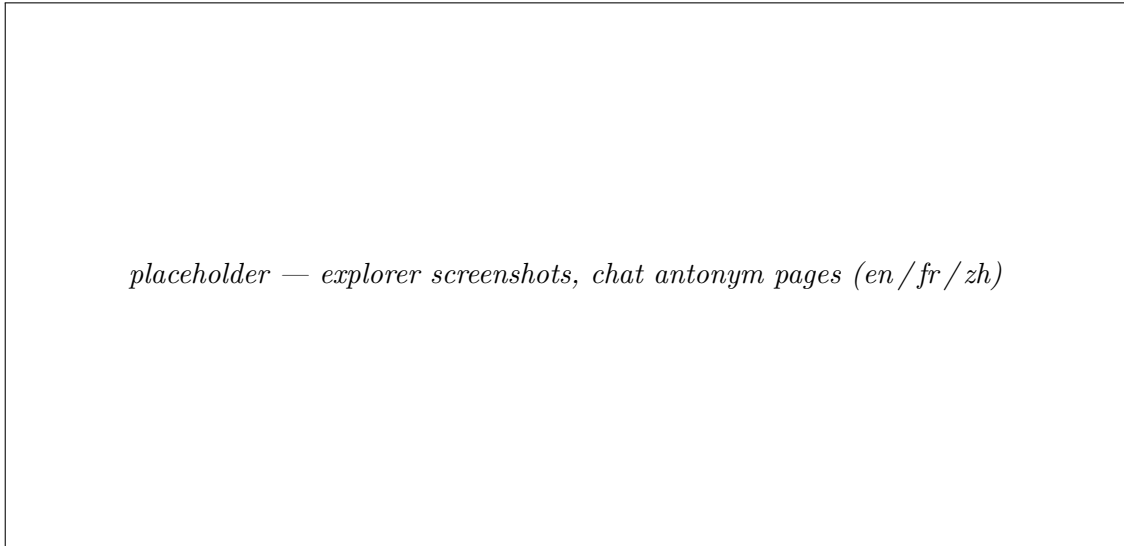


Figure 6: Explorer views of the chat-format antonym graphs with the reviewed supernodes as groups.

placeholder — explorer screenshots, raw antonym pages (en/fr/zh)

Figure 7: Explorer views of the raw open-quote antonym graphs (the paper’s format).